KART: Natural-Language Trajectory Queries over Key-Value Stores

Anonymous Authors

Abstract. Trajectory queries over key-value stores on nature language require query systems to translate user intent into efficient index access while preserving complete result coverage. Existing methods typically rely on the user's expertise and the query plan design capabilities of traditional key-value databases themselves; at the same time, few of today's mainstream key-value databases natively support a Cost-Based Optimizer (CBO) similar to those found in relational databases. This often makes it difficult to perform efficient queries on key-value databases. We present KART, a natural-language trajectory query system for key-value stores. KART maps requests to a typed intermediate representation, performs bounded cost-guided search over legal access plans, and uses offline LLM assistance to curate rewrite rules and calibrate cost estimates. At the same time, a validator checks the execution plan before execution to ensure the correctness of the results and semantics. We implement KART on HBase and evaluate it on the T-Drive and Porto datasets using translation, query-processing, optimizer, and ablation workloads. Experiments on real-world datasets show that KART significantly reduces query latency and improves optimization efficiency for queries of varying complexity. Overall, KART outperforms the best-performing baseline by up to TBD%.

Keywords: TBD

1 Introduction

Trajectory data represents object movement as sequences of timestamped locations and supports mobility analysis, route retrieval, and regional activity studies. Key-value (KV) stores such as HBase provide scalable storage through RowKey lookups and ordered range scans. A trajectory query system must map spatial, temporal, and attribute predicates to these physical access mechanisms, then refine retrieved candidates to preserve the query semantics. Its performance and correctness therefore depend on data layout, index organization, and access-plan selection.

Large language models (LLMs) make it possible to express database requests in natural language. xx et al. have studied text-to-SQL and text-to-NoSQL approaches, and there are also numerous related works on LLM-based query optimization, such as: TBD, TBD However, these systems and methods have not yet addressed physical-layer issues related to trace queries on key-value (KV) stores, including RowKey ranges, dedicated trace indexes, and block-level candidate operations. This gives rise to the following three challenges.

Natural-language translation. Requests may omit or ambiguously express temporal, spatial, attribute, or similarity constraints. A system must resolve intent against supported

Figure placeholder

TBD: Diagrams should be provided for the three challenges.

Specifically, there are three subdiagrams—a, b, and c—which should have similar structures and correspond to the three challenges, respectively. Challenges 2 and 3 occur during the query phase, while Challenge 1 occurs during the processing phase.

Fig. 1. TBD

trajectory semantics and its logical catalog, and request clarification when required information is missing.

Access-path selection. A query may use or combine temporal, spatial, and attribute indexes, followed by candidate operations and exact filtering. The best path depends on both query selectivity and the costs of scans, retrieval, and refinement; fixed rules may not adapt to these differences.

Plan correctness. A syntactically valid plan can still omit a relevant index range, change query semantics, or compile to an invalid HBase request. Exact filtering cannot recover records missed during access, so plans require validation for semantics, candidate coverage, and physical safety.

To address these limitations, we present KART, a natural-language trajectory query system for KV stores. It maps requests through schema-constrained DraftIR and deterministically bound BoundIR representations, searches a bounded space of legal access plans using cost estimates, and validates compiled candidates before execution. Two offline LLM-assisted paths curate scoped rewrite rules and calibrate plan-cost estimates; runtime proposals remain constrained by the legal operator space, and the cost-based path remains available when a rewrite is rejected or unavailable. In summary, we make the following contributions:

- We design an end-to-end query framework that separates language interpretation from binding and physical access planning through typed DraftIR and BoundIR representations.
- We develop bounded, cost-guided access-path search with two offline LLM-assisted artifacts: conditional rewrite rules for candidate generation and cost patches for plan ranking.
- We have built a multi-layered correctness assurance mechanism. The verification process covers the entire workflow, from semantics to concrete plans. Only plans that pass verification are compiled into HBase operations.
- We conducted extensive experiments on two real-world datasets of different sizes, demonstrating KART's query efficiency and robustness across various scenarios and levels of complexity.

2 Preliminaries and Problem Definition

2.1 Definitions

Definition 1 (Trajectory). A trajectory $T_i = \langle p_{i,1}, \ldots, p_{i,m_i} \rangle$ is an ordered sequence of spatio-temporal points, where $p_{i,j} = (x_{i,j}, y_{i,j}, t_{i,j})$. Its minimum bounding rectangle, $\mathrm{MBR}(T_i)$, is the smallest axis-aligned rectangle enclosing its locations.

Definition 2 (Trajectory Query Types). Let $\mathcal{T}$ be a trajectory dataset, W_s a spatial window, W_t a temporal window, T_q a reference trajectory, $d(\cdot,\cdot)$ a trajectory distance, and τ a distance threshold. The query types are defined below.

Spatial range query (SRQ). An SRQ returns trajectories with a location in W_s. Writing $\mathrm{loc}(p)$ for the spatial component of p, we have

$$\mathcal{T}_{\mathrm{SRQ}}(W_s) = \{T_i \in \mathcal{T} \mid \exists p \in T_i : \mathrm{loc}(p) \in W_s\}. \tag{1}$$

Spatio-temporal range query (ST-range). An ST-range requires a location and its timestamp to satisfy the two windows jointly:

$$\mathcal{T}_{\mathrm{ST}}(W_s, W_t) = \{T_i \in \mathcal{T} \mid \exists p \in T_i : p \in W_s \times W_t\}. \tag{2}$$

Similarity search (SS). Given T_q and τ, an SS returns trajectories whose distance to T_q does not exceed τ:

$$\mathcal{T}_{\mathrm{SS}}(T_q, \tau) = \{T_i \in \mathcal{T} \mid d(T_i, T_q) \le \tau\}. \tag{3}$$

Top-k query. Given T_q and $1 \le k \le |\mathcal{T}|$, a Top-$k$ query returns the k closest trajectories:

$$\mathcal{T}_{\text{Top-}k}(T_q) = \arg\min_{\substack{S \subseteq \mathcal{T} \\ |S|=k}} \sum_{T_i \in S} d(T_i, T_q). \tag{4}$$

TBD: the reason why KART use SRQ as workload experiments.

Definition 3 (Logical Query Plan). A logical plan π is an operator graph that retrieves candidate chunks, refines them against the query, and forms the requested result. Its evaluation is written as

$$[\![\pi]\!]_{\mathcal{D}}(q) = \mathrm{Post}_q\big(\mathrm{Refine}_q\big(\mathrm{Fetch}_{\mathcal{D}}\big(\mathrm{Combine}\big(\mathrm{Scan}_{a_1}, \ldots, \mathrm{Scan}_{a_r}\big)\big)\big)\big), \tag{5}$$

where each Scan_{a_j} uses access path a_j, Combine merges candidate chunks, and Post_q applies any query-specific similarity or Top-k operation.

Definition 4 (Safe Query Plan). A logical plan π is safe if its access operators retrieve every chunk that may contain a query match. Let $C^\star(q, \mathcal{D})$ be those required chunks and $C_\pi(q, \mathcal{D})$ the chunks retrieved by π. Coverage safety requires

$$C^\star(q, \mathcal{D}) \subseteq C_\pi(q, \mathcal{D}). \tag{6}$$

TBD: in ablation experiments will discuss why without verify is ok

2.2 Query Optimization

Access paths and plan alternatives. Access paths and execution plans. Access paths map query predicates to key ranges or equality lookups. An execution plan may use a single index, combine the results of compatible indexes, or scan the raw data. Kart provides a variety of indexes, including advanced indexes such as TShape and XZ*.

Rule-based optimization enforces a fixed access-path order. Conversely, cost-based optimization evaluates estimated latencies, recognizing that while index scans reduce candidate refinement, their scanning and set-operation overhead means the most selective path does not always yield the lowest latency.

Cost model. Let $F_{\boldsymbol{\theta}}(\mathbf{x})$ estimate a plan's execution latency from features $\mathbf{x}$ and calibrated coefficients $\boldsymbol{\theta}$. The estimate combines index access, set operations, data retrieval, exact filtering, trajectory reconstruction, similarity computation, and Top-k selection:

$$\begin{aligned}\widehat{L}_{\text{exec}} = \widehat{L}_{\text{index}} + \widehat{L}_{\text{set}} + \widehat{L}_{\text{fetch}} + \widehat{L}_{\text{exact}} \\ + \widehat{L}_{\text{reconstruct}} + \widehat{L}_{\text{sim}} + \widehat{L}_{\text{topk}}.\end{aligned} \tag{7}$$

These metrics are derived from the execution plan, snapshot statistics, and sample estimates. In distributed environments, these metrics also take into account concurrency and the capacity of shared RegionServers. Coefficient-related terms that are not used are set to 0. Later, in the "TBD" section, we discuss work on calibrating the coefficients using an offline LLM.

Fast Cost and Final Cost. Both estimators use the same model and the feature vector $\mathbf{x}$ from Eq. (7):

$$\begin{aligned}\widehat{L}_{\text{Fast}}(\pi) &= F_{\boldsymbol{\theta}}(\mathbf{x}(\pi, q, \mathcal{D})), \\ \widehat{L}_{\text{Final}}(\pi) &= F_{\boldsymbol{\theta}}(\mathbf{x}(\mathcal{K}(\pi), q, \mathcal{D})).\end{aligned} \tag{8}$$

During search, Fast Cost uses the plan, query, and snapshot to estimate ranges and candidate counts. Final Cost uses the compiled access information to refine access and scheduling costs; row and candidate counts remain estimates, so it ranks plans without predicting exact runtime.

Planning objective. For bound query q and snapshot $\mathcal{D}$, let $\Pi^{B}_{\text{safe}}(q, \mathcal{D})$ be the safe candidates reached within planning budget B. Cost-based selection seeks

$$\widehat{\pi} = \arg \min_{\pi \in \Pi^{B}_{\text{safe}}(q, \mathcal{D})} \widehat{L}_{\text{Final}}(\pi). \tag{9}$$

Kart uses this cost model for baseline search and constrained rewriting decisions; Its offline LLM-assisted cost patching and rewriting rules will be detailed later in the TBD section.

2.3 Problem Definition and Objective

Given a natural-language request u and a pinned snapshot $\mathcal{D}$, Kart binds u to a query q and selects a safe plan from the candidates reached within budget B, minimizing estimated execution cost as in Eq. (9). Only plans that pass validation are executed, and coverage safety ensures that exact refinement can produce the complete result for q.

3 Overview

KART supports natural-language trajectory queries by coordinating dataset indexing, bounded access-plan optimization, and deterministic validation. These components turn user requests into efficient HBase operations while preserving query semantics and complete result coverage. Figure 2 summarizes the end-to-end workflow.



Fig. 2. Overview of the KART architecture.

1. Dataset preparation and optimizer configuration. KART normalizes each input dataset into trajectory chunks, builds the configured indexes, and derives statistics for plan costing. Historical workload evidence is also used offline to prepare scoped rewrite rules and cost patches.
2. Natural-language interpretation and plan generation. Given a request and its dialogue context, the query interpreter produces a typed representation; a deterministic binder resolves its values against the catalog and selected dataset snapshot. The planner then searches a bounded space of legal access plans, and the compiler maps retained candidates to HBase scan tasks.
3. Plan validation and query execution. Before execution, a deterministic validator checks that each compiled plan preserves query semantics, covers all possible matches, and uses safe HBase scan ranges. The executor runs an accepted plan, applies exact predicates to retrieved trajectory data, and returns the requested result.

4 Methodology

4.1 Constrained Natural-Language Translation to BoundIR

Generating HBase scans directly from natural language would mix uncertain interpretation with storage-specific decisions, risking unsupported fields or incomplete key ranges. KART instead limits the model to a typed logical request and delegates binding to deterministic code.

Given the request and dialogue context, the model receives the logical catalog, supported semantics, and DraftIR schema, and returns logical predicates, attributes,

similarity options, and result mode. Schema validation excludes undeclared fields and physical storage details. The parser may request up to two repairs; missing facts trigger clarification, while unsupported semantics are rejected.

The binder canonicalizes temporal and spatial values, resolves registered regions and external trajectory identifiers, attaches the selected dataset manifest and semantics version, and checks cross-field constraints. It produces

$$q = \mathrm{Bind}(d, C, \mathcal{D}), \tag{10}$$

where d is DraftIR, C the logical catalog, and $\mathcal{D}$ the pinned dataset snapshot. Thus, the optimizer receives a complete BoundIR, while physical plan construction remains under system control.

4.2 Offline LLM Assistance for Query Optimization

Cost-based optimization is a natural baseline: it enumerates supported access plans and selects the one with the lowest estimated cost. Kart retains this CBO path. However, cost ranking alone cannot use historical query-plan evidence to add workload-specific rewrites, and a generic estimate may mis-rank plan families on a particular dataset and layout. Kart therefore augments CBO with two offline LLM-assisted mechanisms: admitted rewrite rules adjust a baseline plan using supported transformations, while a cost patch calibrates plan-family estimates. The LLMs prepare these artifacts offline; they do not replace the runtime optimizer.

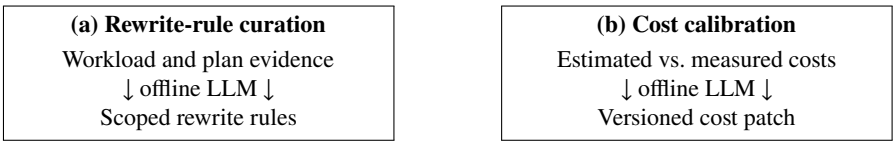


Fig. 3. Offline LLM assistance produces separate artifacts for plan generation and cost ranking.

Conditional rewrite-rule curation. The rule assistant addresses a discrete, evidence-sparse problem: which supported access or resource choice should change for a recurring query-plan context? A learned policy would require a representative labeled workload for each layout and could generalize a choice to contexts for which no evidence exists. Reinforcement learning would additionally require repeated reward-driven exploration, whose execution cost is high and whose proposed plans still need independent correctness checks. Kart instead uses the LLM to summarize measured cases into an auditable rule proposal; a deterministic admission gate, rather than the LLM, decides whether the proposal can be used.

Formally, let $\mathbf{z}(q, \pi, \mathcal{D})$ be the available query, CBO-plan, and snapshot features. A proposed rule is a triple $r = (\sigma, \phi, \delta)$: σ is its dataset/layout/semantics/statistics scope, ϕ is a conjunction of feature predicates, and δ assigns values only to enumerated access

or resource slots. Its application is

$$r(\pi) = \begin{cases} \mathrm{Build}_{\mathrm{legal}}(\pi, \delta), & \mathcal{D} \models \sigma \ \wedge \ \phi(\mathbf{z}) = 1, \\ \pi, & \text{otherwise.} \end{cases} \tag{11}$$

The rule cannot introduce an operator or physical key outside the constrained builder's catalog. Admission requires a confirmed measured source, a source fingerprint, and support from at least three cases; the scope is checked against the active manifest before matching. Thus, the LLM supplies a compact conditional hypothesis, while evidence, scope, and the legal action schema bound its use. A matching rewrite becomes an additional candidate and is compiled and validated; the original CBO plan remains available as fallback.

Plan-cost patch calibration. Candidate generation and candidate ranking are separate: a useful rewrite can still be missed if the base cost model misranks its plan family. For each query q, the calibration input therefore groups plan pairs $(\pi, \widehat{L}(\pi), L(\pi))$, where $\widehat{L}$ is the estimated latency and L is the measured execution latency. A cost model trained from scratch would need a larger representative sample and can overfit when layouts or workloads change. The LLM instead examines the residual ranking errors and proposes a small set of plan-family corrections; a bounded-scale check and measured-pair comparison determine whether to retain them. The patch is frozen with its dataset scope, so runtime costing remains deterministic.

Let Q_{cal} be the calibration queries, Π_q their measured plans, and $\mathcal{J}$ the cost components in Eq. (7). The patch uses one scale s_f per eligible plan family:

$$\widehat{L}_{\mathrm{patch}}(\pi) = s_{f(\pi)} \sum_{j \in \mathcal{J}} \widehat{L}_j(\pi), \quad s_f \in [s_{\min}, s_{\max}], \quad s_f = 1 \text{ when unpatched.} \tag{12}$$

For an evaluated scale vector $\mathbf{s}$, define the estimated and measured winners for query q as $\hat{\pi}_q(\mathbf{s}) = \arg\min_{\pi \in \Pi_q} \widehat{L}_{\mathrm{patch}}(\pi)$ and $\pi_q^{\mathrm{m}} = \arg\min_{\pi \in \Pi_q} L(\pi)$. The offline loop selects among LLM-proposed candidates by

$$\begin{aligned} \mathbf{s}^* &= \operatorname*{lexmax}_{\mathbf{s} \in \mathcal{S}_{\mathrm{eval}}} \big(A(\mathbf{s}), -M(\mathbf{s})\big), \\ A(\mathbf{s}) &= \frac{1}{|Q_{\mathrm{cal}}|} \sum_{q \in Q_{\mathrm{cal}}} \mathbb{I}[\hat{\pi}_q(\mathbf{s}) = \pi_q^{\mathrm{m}}], \\ M(\mathbf{s}) &= \mathrm{median}_{q \in Q_{\mathrm{cal}}} L(\hat{\pi}_q(\mathbf{s})). \end{aligned} \tag{13}$$

where $\mathcal{S}_{\mathrm{eval}}$ is the set of evaluated, bounded proposals and lexicographic order prioritizes winner agreement A, then the median measured latency M of selected plans. The LLM receives the largest remaining disagreements to revise eligible plan-family scales; configured bounds and frozen families constrain each proposal. Only the selected scales are serialized. Equation (12) changes plan ranking only; it does not change query predicates, access ranges, or coverage.

Bounded search using offline artifacts. The two offline outputs have complementary roles in one runtime search: rewrite rules determine which evidence-backed alternatives can be added to the CBO candidate space, whereas the cost patch adjusts how those alternatives are ranked. CBO still supplies the baseline plan and legal access-plan space. For BoundIR q, snapshot $\mathcal{D}$, and search budget B, let Π_B be the plans reached by bounded expansion and $\mathcal{S}_B$ the compiled plans accepted by the validator. The runtime objective is

$$\pi^* = \arg\min_{\pi \in \mathcal{S}_B} \widehat{L}_{\text{patch}}(\pi), \qquad \mathcal{S}_B = \{\pi \in \Pi_B : \text{Valid}(\pi, q, \mathcal{D})\}, \tag{14}$$

with the unpatched CBO estimate used when no matching patch exists. FullScan remains an alternative. Legal access changes and admitted rewrites are expanded under the configured beam, candidate, proposal-call, and planning-time limits; Fast Cost ranks partial plans, and the compiled-plan estimate is used for final ranking. The validator therefore gates the candidate set before cost decides among it. Kart returns the lowest estimated-cost safe plan reached within B, not a guarantee of the global optimum. Algorithm 1 summarizes this integration.

Algorithm 1 Bounded search and safe-plan selection

Require: BoundIR q, snapshot $\mathcal{D}$, policy ρ, budget B
Ensure: Query result or explicit failure
$\mathcal{C} \leftarrow \{\pi_{\text{full}}\}$; $\mathcal{F} \leftarrow$ Initialize$(q, \mathcal{D})$
while $\mathcal{F} \neq \emptyset$ and B remains **do**
  $\mathcal{A} \leftarrow$ LegalActions$(\mathcal{F}, q, \mathcal{D})$
  $(\mathcal{C}', \mathcal{N}) \leftarrow$ ExpandAndCheck$(\mathcal{F}, \mathcal{A}, \rho, B)$
  $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$; $\mathcal{F} \leftarrow$ KeepBest$(\mathcal{N}, B.\text{beamWidth})$
end while
$\mathcal{S} \leftarrow$ CompileAndValidate$(\mathcal{C}, q, \mathcal{D})$
if $\mathcal{S} = \emptyset$ **then return** NoSafePlan
end if
if accepted direct plan $\pi_d \in \mathcal{S}$ is selected **then**
  $\pi^\star \leftarrow \pi_d$
else
  $\pi^\star \leftarrow \arg\min_{\pi \in \mathcal{S}} \widehat{L}_{\text{Final}}(\pi)$
end if
return CoordinatorExecute$(\pi^\star)$

4.3 Plan Validation for Semantic and Coverage Safety

A plan is eligible for cost-based selection only when both its logical meaning and compiled HBase reads pass explicit predicates. For a candidate π, BoundIR q, snapshot $\mathcal{D}$, and compiled physical plan $K(\pi)$, define

$$\text{Valid}(\pi, q, \mathcal{D}) = V_{\text{str}}(\pi) \wedge V_{\text{sem}}(\pi, q) \wedge V_{\text{cov}}(K(\pi), q, \mathcal{D}) \wedge V_{\text{phys}}(K(\pi), \mathcal{D}). \tag{15}$$

The checks are ordered so malformed plans are rejected before more expensive range checks. V_{str} requires a schema-conforming, well-typed, acyclic operator graph with valid

references and arities. V_{sem} checks that the graph's filters and result operators implement the predicates and result mode in BoundIR. V_{cov} checks the emitted ranges against the required chunk set,

$$C^{\star}(q, \mathcal{D}) \subseteq C_{K(\pi)}(q, \mathcal{D}), \tag{16}$$

and V_{phys} requires every scan to target a manifest-declared table and shard with valid, non-truncated bounds. In particular, coverage is established from compiled ranges: all required temporal buckets, intersecting quantized spatial cells, equality-key shard prefixes, and each branch of an intersection must be represented. This is the key correctness condition because exact refinement can discard false positives but cannot recover a missing chunk. A `SafePlanHandle` is issued exactly when all four predicates pass; if no candidate passes, Kart fails closed rather than executing an unverified plan.

5 Experiments

5.1 Experimental Settings

Datasets.

T-Drive. The dataset contains 318,774 pre-segmented Beijing taxi trajectories collected over one week. For scalability, we construct ×1, ×10, ×50, and ×100 variants by replicating trajectories with unique identifiers and shifting each replica's timestamps by one week; spatial coordinates are unchanged.

Porto. The dataset contains approximately 250,000 pre-segmented taxi trajectories collected over two months. Its scale variants are trajectory-level random subsamples at ×1, ×0.5, ×0.25, and ×0.1 (approximately 250,000, 125,000, 62,500, and 25,000 trajectories), generated with a fixed seed. Thus, T-Drive scaling expands the data, whereas Porto scaling measures performance on smaller samples.

Query Workloads.

NL-to-IR. The frozen T-Drive set contains 40 natural-language requests: 20 simple and 20 complex. It covers SRQ, ST-range, similarity search (SS), and Top-k, with five requests per query type and difficulty; basic and POI-oriented requests are both represented.

Query-plan comparison. The frozen BoundIR workloads contain 24 simple and 24 complex queries on each dataset. Each difficulty level is balanced across the same four query types (six queries per type) and is evaluated against its corresponding FullScan oracle.

Window-distribution study. To isolate spatial workload placement, we generate SRQ workloads under uniform, Gaussian, and skewed distributions while holding each dataset snapshot fixed. Each distribution contains ten query centers and five nested square-window widths (50, 100, 500, 1,000, and 1,500 m), with a four-hour temporal window. Uniform centers are spread over the dense spatial core; Gaussian centers cluster around its densest hotspot; skewed centers select among eight hotspots with Zipf weights (95% of samples) and use uniform placement for the remainder. The same six query-processing methods are compared on both datasets.

Baselines.

NL-to-IR. DirectIR prompts a language model to emit the target IR directly. DIN-SQL (Spider/BIRD) adapts the corresponding Text-to-SQL pipelines to the query schema, while SAG-MQL uses schema-path grounding followed by MQL generation.

Query planning and execution. FullScan reads all raw chunks and provides the answer oracle. RBO selects access paths by a fixed priority; CBO ranks the classic access alternatives with rule-based cost estimates. Bao supplies learned plan-choice guidance, and ParamTree supplies learned cost estimates; both are adapted to the same BoundIR interface and HBase execution path.

Metrics and Parameter Settings. We evaluate translation action accuracy and query-processing latency, with correctness checked against FullScan. Table 1 summarizes the beam and MC settings and the supported query types.

Implementation Details. All plan and execution comparisons use frozen BoundIR inputs rather than generated NL-to-IR outputs, so translation errors do not affect optimizer measurements. For each query and dataset, every method receives the same snapshot and query; its result is compared with the FullScan oracle (set equality for identifier queries and ordered-ID equality for Top-k). The E1 translation suite is run separately against its expected action and gold IR. Runs use HBase 2.2.3, Hadoop 2.8.5, ZooKeeper 3.4.10, OpenJDK 8, and four RegionServers. The advanced indexes are enabled for workloads that use them; caches are warmed before timed repetitions.

5.2 Overall Performance

We evaluate translation quality, workload sensitivity, and query planning and execution. The NL-to-IR results summarize E1, the distribution table characterizes SRQ workloads, and the E2/E3 plots compare planning and end-to-end performance across query types, difficulty levels, and datasets.

Workload results. Table 2 reports end-to-end latency for the uniform, Gaussian, and skewed SRQ workloads. Each cell is the P50 latency in milliseconds; entries without recorded measurements are marked *TBD*.

NL-to-IR (E1). Table 3 presents the expected-action accuracy and P50 parse latency for simple and complex requests. Action accuracy is shown separately for Pass, Reject, and Clarify; Pass additionally requires semantic equivalence to the gold IR.

Table 1. Query and search parameter settings.

Parameter	Setting
Beam	$\{1, 3, 5, 7\}$ (default: 3)
MC	$\{4, 6, 8, 10\}$ (default: 4)
Query type	SRQ, ST-range, SS, Top-k

Table 2. End-to-end latency (P50, ms) by spatial workload distribution.

Dataset	Distribution	KART	FullScan	RBO	CBO	Bao	ParamTree
T-Drive	Uniform	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
	Gaussian	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
	Skewed	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Porto	Uniform	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
	Gaussian	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
	Skewed	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*

Table 3. E1 NL-to-IR results on T-Drive. Action columns report Pass/Reject/Clarify accuracy; latency is P50 in milliseconds.

	Simple queries		Complex queries	
Method	Action accuracy	P50	Action accuracy	P50
KART	1.00/1.00/1.00	1,562	1.00/1.00/1.00	1,596
DirectIR	0.7778/0.3333/0	1,586	0.7778/0.3333/0	2,033
DIN-SQL (Spider)	0.3333/0.3333/0	14,151	0.6667/0.5000/0	14,097
DIN-SQL (BIRD)	0.6667/0.3333/0	18,292	0.5556/0.5000/0	8,272
SAG-MQL	0.6667/0.5000/0	1,329	0.7778/0.3333/0	1,789

Query planning and execution (E2/E3). E2 measures planning latency from frozen BoundIR inputs; E3 measures planning, validation, and physical execution end to end. The six panels in Fig. 4 compare the methods by dataset and report planning and execution behavior without aggregating the simple and complex workloads together. All query outputs are checked against FullScan.

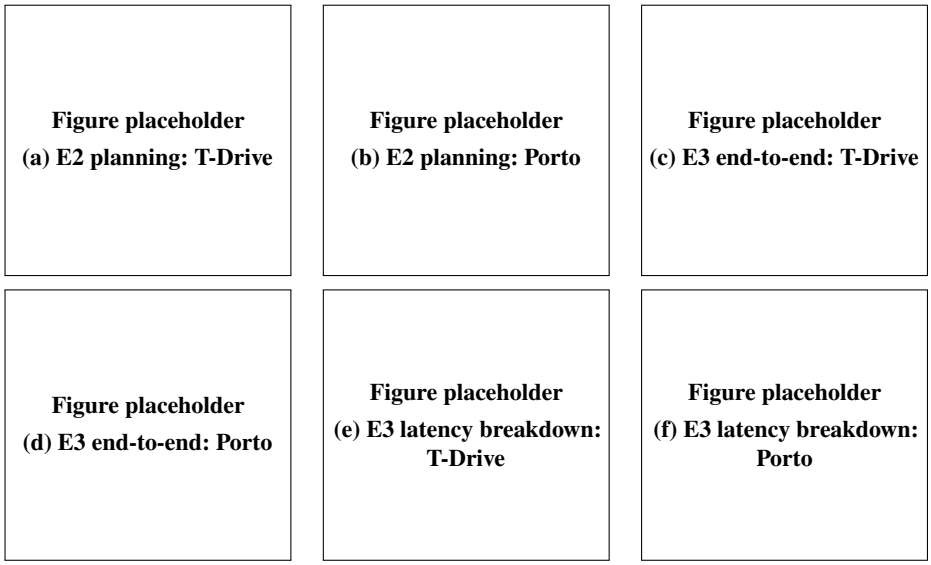


Fig. 4. Planning and end-to-end performance across datasets and workload difficulty.

5.3 Ablation Study

The ablation compares full KART with variants that remove the rewrite-rule LLM, the cost-model LLM, the cost model, or plan verification. The results show exact-match agreement of 1.00 for all tested variants. Removing the cost model causes the largest E2E increase, particularly on Porto complex queries; the other removals have smaller, workload-dependent effects.

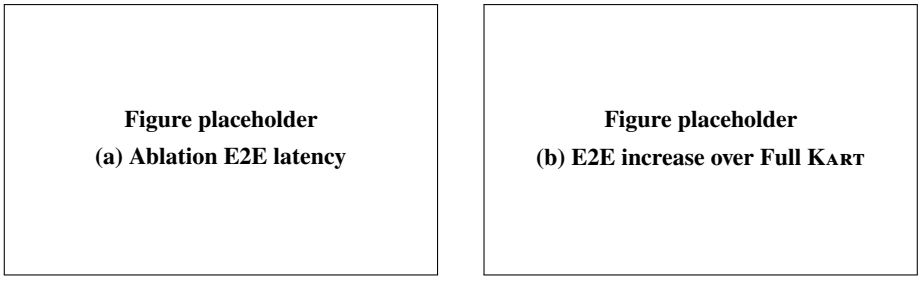


Fig. 5. Ablation results across datasets and workload difficulty.

Rewrite-rule LLM. Removing conditional rewrite rules increases E2E latency most clearly on T-Drive, where cost-guided candidates lose useful access-path refinements. The smaller Porto change suggests that the benefit depends on how well the recorded rules match a dataset's query and layout patterns.

Cost-model LLM. Without offline cost calibration, plan ranking becomes less aligned with observed execution cost. The increase is modest on T-Drive but pronounced for complex Porto queries, where selecting an expensive plan can dominate E2E latency.
Cost model. Removing cost estimates has the largest impact. Fixed or weakly differentiated choices cannot account for scan, candidate-fetch, and refinement costs, so the selected access path can perform substantially more execution work.
Plan verification. Disabling verification does not consistently lower or raise E2E latency, and the tested outputs remain exact. Verification instead enforces semantic, coverage, and physical-safety conditions that timing results alone do not establish.

5.4 Parameter Study

We study two search-budget parameters. Beam width (beam) limits the number of partial plans retained at each search step, while the candidate limit (MC) bounds the number of completed plans considered. We report E2E latency only. In the beam sweep, MC is fixed at 8; in the MC sweep, beam is fixed at 3. The configured defaults are beam 3 and MC 4.

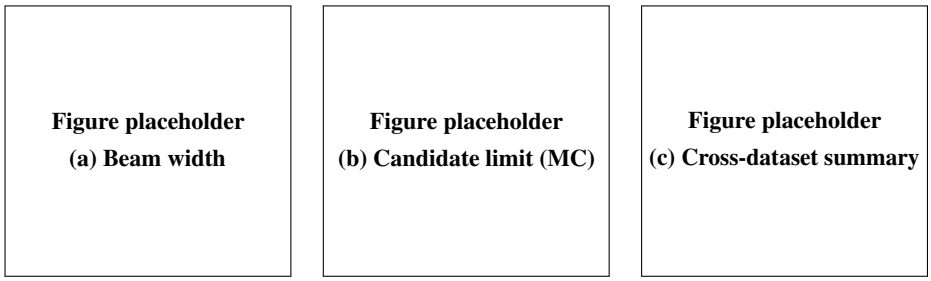


Fig. 6. E2E latency under different bounded-search parameter settings.

For beam width, T-Drive reaches its lowest recorded E2E latency at beam 3 (195/220 ms for simple/complex queries); wider beams do not yield consistent gains (199–205/232–237 ms). Porto is less sensitive, with recorded values between 1,043 and 1,060 ms for simple queries and 1,043 and 1,057 ms for complex queries. This suggests that retaining a wider frontier adds search effort without reliably improving the selected plan.

For MC, T-Drive improves as the limit grows from 4 to 8 (207/239 to 195/220 ms), then regresses at 10 (206/244 ms), indicating a useful balance between considering more alternatives and limiting planning overhead. Porto varies less monotonically: the simple workload is lowest at MC 10 (1,028 ms), while complex latency is similar at MC 4 and 10 (1,043 ms) and slightly higher at intermediate settings. Thus, the best MC value depends on dataset and workload difficulty; the recorded changes are in E2E latency only.

5.5 Scalability

We measure E2E latency across T-Drive expansions ($\times 1$, $\times 10$, $\times 50$, $\times 100$) and Porto trajectory subsamples ($\times 1$, $\times 0.5$, $\times 0.25$, $\times 0.1$). Each scale uses the same query semantics

and compares KART with the five IR-level baselines. The measured latency and index-footprint results are *TBD*.

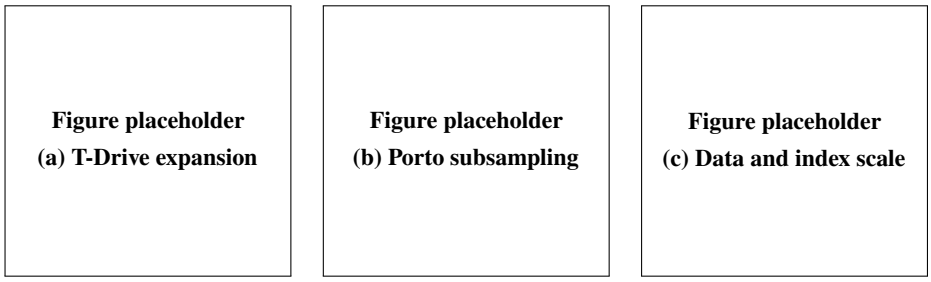


Fig. 7. Query latency and data/index size across dataset scales.

The expected T-Drive trend is increasing E2E latency with larger replicas, since more rows and postings increase index scans and chunk retrieval. With fixed query windows and spatial coordinates, however, the candidate set need not grow in proportion to the replicated dataset; indexed plans should therefore scale more gently than FullScan. For Porto, smaller samples reduce the indexed population, candidate chunks, and exact-refinement work, so latency is expected to fall as the sample fraction decreases. At the smallest scales, fixed planning and scan startup costs may limit further gains. The measured trends are *TBD*; these effects explain the expected dataset-specific behavior.

5.6 Representative Query Case Study

TBD

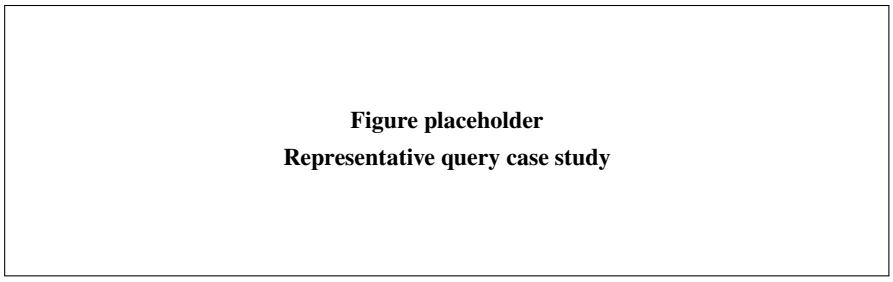


Fig. 8. Case-study query, selected access plan, and result.

6 Related Work

6.1 Trajectory Indexing for Key-Value Stores

Early systems linearize spatial locations to support ordered-key scans: UlTraMan applies space-filling-curve encoding to trajectory data, and TrajMesa builds on XZ-order-based keys for spatial retrieval [1,4]. Later work improves how a trajectory or region is represented in those keys. TraSS uses trajectory-shape encoding to prune candidates for similarity search, while TMan introduces richer spatial encodings, including XZ* region combinations and TShape multi-scale shape partitions [2,3]. These indexes reduce scan ranges or candidate volume, and are available as candidate sources in KART; KART focuses on selecting among such access paths while preserving coverage.

6.2 Natural-Language Interfaces to Databases

Text-to-SQL work first improved generation by decomposing the task: DIN-SQL separates schema linking from SQL construction and adds self-correction, while DTS-SQL further studies decomposed generation with smaller language models [7,8]. More recently, TEND extends natural-language querying to NoSQL and evaluates execution-verified Text-to-NoSQL translation; its SAG method grounds schema paths using stored data [5]. Together, these methods progress from structured-query generation toward schema grounding for non-relational stores.

6.3 Learning-Based Query Optimization

Learned optimization has progressed from guiding plan choices to comparing plans and improving cost estimates. Bao uses learned guidance over optimizer-generated plans [6]; Lero learns to rank alternative plans directly [10]; and ParamTree develops a learned cost model for query optimization [9]. These approaches use workload or execution evidence to improve plan selection. KART adapts cost guidance to key-value access plans, whose costs depend on scans, index intersections, and candidate retrieval, while validating plan safety independently.

7 Conclusion

This paper introduces a natural language trajectory query system for key-value stores called Kart. Kart maps user requests to a typed intermediate representation, utilizes cost guidance and offline large language models (LLMs) to search for candidate access plans, and validates the plans before execution. Experiments on two real-world datasets, T-Drive and Porto, demonstrate Kart's superiority, with query latency reduced by TBD to TBD compared to the strongest benchmarks.

References

1. Ding, X., Chen, L., Gao, Y., Jensen, C.S., Bao, H.: UlTraMan: A unified platform for big trajectory data management and analytics. Proceedings of the VLDB Endowment **11**(7), 787–799 (2018). https://doi.org/10.14778/3192965.3192970
2. He, H., Li, R., Ruan, S., He, T., Bao, J., Li, T., Zheng, Y.: TraSS: Efficient trajectory similarity search based on key-value data stores. In: Proceedings of the IEEE International Conference on Data Engineering. pp. 2306–2318. IEEE (2022). https://doi.org/10.1109/ICDE53745.2022.00218
3. He, H., Xu, Z., Li, R., Bao, J., Li, T., Zheng, Y.: TMan: A high-performance trajectory data management system based on key-value stores. In: Proceedings of the IEEE International Conference on Data Engineering. pp. 4951–4964. IEEE (2024). https://doi.org/10.1109/ICDE60146.2024.00376
4. Li, R., He, H., Wang, R., Ruan, S., Sui, Y., Bao, J., Zheng, Y.: TrajMesa: A distributed NoSQL storage engine for big trajectory data. In: Proceedings of the IEEE International Conference on Data Engineering. pp. 2002–2005. IEEE (2020). https://doi.org/10.1109/ICDE48307.2020.00224
5. Lu, J., Lu, J., Zhang, C., Qin, Z., Zhang, H., Song, Y., Wong, R.C.W.: Bridging the gap: Enabling natural language queries for NoSQL databases through text-to-NoSQL translation. arXiv preprint arXiv:2502.11201 (2025). https://doi.org/10.48550/arXiv.2502.11201, version 3 updated June 2026
6. Marcus, R., Negi, P., Mao, H., Zhang, C., Alizadeh, M., Kraska, T., Papaemmanouil, O., Tatbul, N.: Bao: Making learned query optimization practical. In: Proceedings of the 2021 International Conference on Management of Data. pp. 1275–1288. ACM (2021). https://doi.org/10.1145/3448016.3452838
7. Pourreza, M., Rafiei, D.: DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. arXiv preprint arXiv:2304.11015 (2023). https://doi.org/10.48550/arXiv.2304.11015
8. Pourreza, M., Rafiei, D.: DTS-SQL: Decomposed text-to-SQL with small large language models. In: Findings of the Association for Computational Linguistics: EMNLP 2024. pp. 8212–8220. Association for Computational Linguistics (2024). https://doi.org/10.18653/v1/2024.findings-emnlp.481
9. Yang, J., Wu, S., Zhang, D., Dai, J., Li, F., Chen, G.: Rethinking learned cost models: Why start from scratch? Proceedings of the ACM on Management of Data **1**(4), 1–27 (2023). https://doi.org/10.1145/3626769
10. Zhu, R., Chen, W., Ding, B., Chen, X., Pfadler, A., Wu, Z., Zhou, J.: Lero: A learning-to-rank query optimizer. Proceedings of the VLDB Endowment **16**(6), 1466–1479 (2023). https://doi.org/10.14778/3583140.3583160